

Curso desarrollo de aplicaciones con tecnologías web

UF1842. Desarrollo y reutilización de componentes software y multimedia mediante lenguajes de gui3n.

Lenguajes de gui3n de uso general

Usos espec3ficos de lenguajes de gui3n

Entornos integrados (Frameworks) para el desarrollo con lenguajes de gui3n

Usos específicos de lenguajes de guión

Integración multimedia mediante lenguajes de guión

Los lenguajes de guión, como JavaScript, permiten controlar elementos multimedia en una página web, como audio, video e imágenes, utilizando el DOM.

Ejemplo html:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Control de Video</title>
</head>
<body>
  <video id="miVideo" width="320" height="240" controls>
    <source src="video.mp4" type="video/mp4">
    Tu navegador no soporta el video.
  </video>
  <button onclick="playVideo()">Reproducir</button>
  <button onclick="pauseVideo()">Pausar</button>

  <script>
    let video = document.getElementById("miVideo");
    function playVideo() {
      video.play();
    }
    function pauseVideo() {
      video.pause();
    }
  </script>
</body>
</html>
```

Animaciones

JavaScript permite crear animaciones usando bibliotecas externas como GreenSock (GSAP) o con APIs nativas como requestAnimationFrame.

Ejemplo básico

CSS:

```
#cuadro {  
  width: 50px;  
  height: 50px;  
  background-color: red;  
  position: relative;  
}
```

html:

```
<div id="cuadro"></div>
```

javascript:

```
let cuadro = document.getElementById("cuadro");  
let posicion = 0;  
  
function mover() {  
  if (posicion < 300) {  
    posicion++;  
    cuadro.style.left = posicion + "px";  
    requestAnimationFrame(mover);  
  }  
}  
  
mover();
```

Comparativa: Nativo vs Bibliotecas

Característica	Nativo (requestAnimationFrame)	GSAP	Anime.js	Velocity.js
Rendimiento	Muy alto	Muy alto	Alto	Alto
Facilidad de uso	Requiere más código	Muy fácil	Fácil	Fácil
Soporte para SVG	Limitado	Excelente	Excelente	Bueno
Plugins avanzados	No disponible	Sí (ScrollTrigger, etc.)	No	No
Animaciones complejas	Complejo	Muy sencillo	Sencillo	Moderado

JavaScript ofrece herramientas nativas potentes para animaciones, pero el uso de bibliotecas como GSAP facilita enormemente la creación de animaciones complejas y fluidas, especialmente en proyectos más avanzados.

Efectos especiales en elementos gráficos y texto

Los efectos especiales, como sombreado dinámico o textos que cambian de color, se pueden implementar con CSS combinado con JavaScript.

Ejemplo

CSS:

```
#texto {  
  font-size: 24px;  
  transition: color 0.5s ease;  
}
```

html:

```
<p id="texto">Pasa el ratón sobre este texto</p>
```

javascript:

```
let texto = document.getElementById("texto");  
texto.addEventListener("mouseover", () => texto.style.color = "blue");  
texto.addEventListener("mouseout", () => texto.style.color = "black");
```

Rejillas de datos

Las rejillas de datos se utilizan para mostrar tablas dinámicas. Herramientas como DataTables (jQuery) o Grid.js facilitan su uso.

Ver ejemplo en el repositorio GIT

Manejo de canales de suscripción (RSS)

Se pueden integrar canales RSS con JavaScript, pero normalmente se requiere un backend para obtener y procesar los datos.

Ver ejemplo en el repositorio GIT

Técnicas de acceso asíncrono (AJAX)

AJAX permite actualizar contenido de una página sin recargarla.

Ejemplo:

JavaScript

```
fetch('https://jsonplaceholder.typicode.com/posts/1')  
  .then(response => response.json())  
  .then(data => console.log(data));
```

Tabla con algunos de los métodos y conceptos más relevantes relacionados exclusivamente con AJAX en JavaScript:

Método/Concepto	Descripción	Ejemplo
fetch()	Realiza solicitudes HTTP asíncronas y devuelve una promesa que se resuelve con la respuesta.	<code>fetch('https:// jsonplaceholder.typicode.com/posts/ 1') .then(response => response.json())</code>
XMLHttpRequest	Método más antiguo que fetch() para realizar solicitudes HTTP asíncronas.	<code>let xhr = new XMLHttpRequest(); xhr.open("GET", "url", true);</code>
response.json()	Convierte la respuesta de una solicitud fetch en formato JSON.	<code>fetch('url').then(response => response.json())</code>
response.text()	Convierte la respuesta de una solicitud fetch en texto (útil si no es JSON).	<code>fetch('url').then(response => response.text())</code>
async/await	Sintaxis moderna para manejar promesas de manera más legible y síncrona.	<code>let response = await fetch('url'); let data = await response.json();</code>

Método/Concepto	Descripción	Ejemplo
then()	Se utiliza para manejar la respuesta de una promesa cuando la solicitud es exitosa.	<code>fetch('url').then(response => response.json()).then(data => console.log(data));</code>
catch()	Se utiliza para manejar los errores que ocurren durante la solicitud fetch.	<code>fetch('url').then(response => response.json()).catch(error => console.error('Error:', error));</code>
abort()	Detiene una solicitud HTTP que está en curso con un objeto XMLHttpRequest.	<code>xhr.abort();</code>
setRequestHeader()	Establece los encabezados de la solicitud antes de enviarla (útil para solicitudes POST).	<code>xhr.setRequestHeader("Content-Type", "application/json");</code>
open()	Inicializa una nueva solicitud HTTP con el método (GET, POST, etc.) y la URL.	<code>xhr.open("GET", "https://api.example.com", true);</code>
send()	Envía una solicitud HTTP (utilizado junto con XMLHttpRequest).	<code>xhr.send();</code>

Breve descripción de algunos de los métodos:

1. `fetch()`: Es la forma moderna de hacer solicitudes asíncronas en JavaScript. Devuelve una promesa que puede ser resuelta utilizando `then()` o `await`. Se prefiere sobre XMLHttpRequest por su sintaxis más limpia y facilidad de uso.
2. XMLHttpRequest: Es la antigua forma de hacer solicitudes HTTP asíncronas en JavaScript. Aunque está más compleja que `fetch()`, sigue siendo útil en algunos contextos donde se necesita un control más detallado sobre la solicitud.
3. `then()` y `catch()`: Ambos son métodos de las promesas. `then()` maneja la respuesta exitosa de la solicitud, mientras que `catch()` maneja los errores.

4. `async/await`: Sintaxis más moderna que facilita el trabajo con promesas, ya que permite escribir código asíncrono como si fuera síncrono.

Cada uno de estos métodos te permite interactuar con servicios externos de manera asíncrona, lo que es fundamental para construir aplicaciones web dinámicas que no requieran recargar la página cada vez que se haga una solicitud o actualización de datos.

Uso de lenguajes de guión en dispositivos móviles

Los lenguajes de guión también se usan para mejorar las aplicaciones móviles híbridas usando frameworks como React Native, Ionic o Flutter.

Entornos integrados (Frameworks) para el desarrollo con lenguajes de gui3n

Características de los entornos de uso com3n. Comparativa

Framework	Ventajas	Ejemplo de uso
React	Componentizaci3n, popularidad	SPAs
Angular	Estructura robusta, TypeScript	Aplicaciones empresariales
Vue.js	Simplicidad, f3cil de aprender	Proyectos peque1os

Editores avanzados

Editor	Características destacadas
VS Code	Plugins, autocompletado, depuraci3n integrada
WebStorm	Optimizaci3n para JavaScript

Validaci3n y depuraci3n de c3digo

- Herramientas:
 - Linter como ESLint.
 - Debugger en navegadores.

Técnicas para la documentación del código

- Documentación con JSDoc:

JavaScript

```
/**
 * Suma dos números.
 * @param {number} a - El primer número.
 * @param {number} b - El segundo número.
 * @returns {number} - La suma de a y b.
 */
function sumar(a, b) {
    return a + b;
}
```

Ver <https://jsdoc.app/>

Utilidades adicionales para contenidos dinámicos

Extensiones útiles:

- Redux DevTools.
- Lighthouse (SEO y rendimiento).

Entornos de desarrollo específicos: desarrollo sobre dispositivos móviles

El desarrollo de aplicaciones móviles ha crecido significativamente, y con ello, han surgido herramientas y entornos especializados que facilitan la creación de aplicaciones tanto para iOS como para Android. Entre las herramientas más destacadas se encuentran React Native y Flutter.

- **React Native.**

React Native es un framework de desarrollo móvil basado en React (una biblioteca de JavaScript para la construcción de interfaces de usuario). Permite escribir aplicaciones móviles utilizando JavaScript y React, pero con la ventaja de poder ejecutar el código nativo en dispositivos iOS y Android.

- **Flutter.**

Flutter, creado por Google, es otro framework para el desarrollo de aplicaciones móviles, pero con la particularidad de usar Dart, un lenguaje de programación propio de Google. Flutter permite crear aplicaciones móviles nativas para iOS y Android con un único código base, y su principal ventaja radica en que permite un control total sobre el diseño, sin depender de los widgets predeterminados del sistema operativo.

Ejercicios Prácticos

1. Animaciones y transiciones

Código base

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejercicio de Animaciones y Transiciones</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
      background-color: #f4f4f4;
    }

    #contenedor {
      text-align: center;
    }

    #cuadro {
      width: 100px;
      height: 100px;
      background-color: #007bff;
      margin: 20px auto;
      transition: transform 0.5s ease, background-color 0.5s ease;
```

```
}

#cuadro.animado {
  transform: translateX(200px) rotate(45deg);
  background-color: #28a745;
}

button {
  padding: 10px 20px;
  font-size: 16px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

button:hover {
  background-color: #ddd;
}
</style>
</head>

<body>
  <div id="contenedor">
    <h1>Ejemplo de Animaciones y Transiciones</h1>
    <div id="cuadro"></div>
    <button onclick="activarAnimacion()">Activar Animación</button>
  </div>

  <script>
    function activarAnimacion() {
      const cuadro = document.getElementById("cuadro");
      cuadro.classList.toggle("animado");
    }
  </script>
```

```
</body>
```

```
</html>
```

Explicación del código

1. HTML:

- El contenedor central incluye un cuadro (div) y un botón.
- Al hacer clic en el botón, se activa la animación mediante la función `activarAnimacion()`.

2. CSS:

- Transiciones:
- Se utiliza `transition` en el cuadro para aplicar cambios suaves en las propiedades `transform` (posición y rotación) y `background-color`.
- Clase `.animado`:
- Cambia la posición del cuadro (`translateX(200px)`) y lo rota 45 grados (`rotate(45deg)`).
- También cambia el color del fondo.

3. JavaScript:

- La función `activarAnimacion()` alterna la clase `.animado` en el cuadro usando `classList.toggle()`.
- Esto permite que el cuadro vuelva a su estado inicial si el usuario hace clic de nuevo en el botón.

Ejercicio práctico:

1. Modifica los valores de la animación:

- Cambia la posición del cuadro (`translateX`) o el ángulo de rotación (`rotate`).

2. Agrega más transiciones:

- Añade cambios en tamaño (`scale`) o en el borde (`border-radius`).

3. Prueba con diferentes eventos:

- Cambia el evento de clic por un `mouseover` o `mouseout` para que la animación ocurra al pasar el ratón.

2. AJAX con Fetch y Manejo de Datos Dinámicos

Descripción del Ejercicio:

Vas a crear una página web que permita consultar información de un API pública, mostrarla en una tabla y agregar funcionalidades dinámicas como filtrar resultados y manejar errores.

Instrucciones

1. Crea un archivo HTML con un campo de entrada (input) para buscar un recurso por un identificador y un botón para consultar los datos.
2. Utiliza la API pública de JSONPlaceholder (<https://jsonplaceholder.typicode.com/posts>) para obtener información.
3. Muestra los datos obtenidos en una tabla dinámica.
4. Si el usuario ingresa un valor inválido o no encuentra datos, muestra un mensaje de error.
5. Agrega un botón para “Cargar Todo”, que muestre todos los registros disponibles.

Requisitos del Ejercicio

1. Buscar por ID:
 - El usuario debe poder introducir un número en el campo de texto.
 - Al hacer clic en “Buscar”, se debe consultar la API para obtener un post específico y mostrarlo en la tabla.
2. Manejar Errores:
 - Si el ID no existe o la API no responde, debe mostrarse un mensaje claro al usuario en la página.
3. Cargar Todo:
 - Al hacer clic en “Cargar Todo”, la tabla debe llenarse con todos los registros disponibles en la API.

4. Opcional:

- Personaliza los estilos de la tabla para mejorar la apariencia.
- Agrega una barra de carga o animación mientras se obtienen los datos.

Objetivo del Ejercicio

- Este ejercicio te permitirá aplicar los conceptos aprendidos sobre AJAX, el uso de `fetch()`, el manejo de errores con `catch()`, y la creación de contenido dinámico en la página. Además, refuerza habilidades para manipular el DOM con JavaScript.